

AWSKRUG Gudi Chapter

슬랙 + EKS 운영 자동화

- ① 슬랙에서 자연어로 EKS 운영하기 (MCP + Bedrock)
- ② AI가 장애를 먼저 만든다 (Kiro + FIS + Q Developer)

발표자 : 이수석 | 2026.06.11

AWSKRUG Gudi | 청중 배포용 핸드아웃

슬랙 + EKS 운영 자동화

① 1번 발표

슬랙에서 자연어로 EKS 운영하기

- ✓ VPC 직접 구축 · EKS 클러스터 (eksctl)
- ✓ Prometheus + Grafana 모니터링
- ✓ Amazon Bedrock Claude Sonnet 4.5
- ✓ Slack Bot → MCP → kubectl 제어
- ✓ /eks 자연어 명령 agentic loop 구조

② 2번 발표

AI가 장애를 먼저 만든다

- ✓ Kiro AI IDE로 FIS 템플릿 자동 생성
- ✓ AWS FIS Pod 강제 종료 카오스 실험
- ✓ Bedrock analyze_chaos_result 분석
- ✓ Q Developer HPA · PDB YAML 개선
- ✓ Grafana 셀프힐링 메트릭 시각화

1번 발표 | STEP 1-2 환경 준비 + STEP 3 VPC 구성

STEP 1-2 환경 준비

- AWS CLI · kubectl · eksctl · Helm · Node.js 설치 후 각 버전 숫자 출력 확인
- `aws configure` → 리전: `ap-northeast-2` · output: `json`

\$ aws sts get-caller-identity → 계정 ID · ARN 출력되면 정상

STEP 3 VPC 구성

- 퍼블릭 서브넷 2개 + 프라이빗 서브넷 2개 (`ap-northeast-2a` · `2c` 각 1개씩)
- Internet Gateway 1개 · NAT Gateway 2개 (AZ당 1개, EIP 각 1개 필요)
- 퍼블릭 서브넷 태그 `kubernetes.io/role/elb=1` → ALB Controller 자동 인식에 필수
- 프라이빗 서브넷 → NAT GW 경유 아웃바운드 (ECR pull · 패키지 설치)

※ NAT Gateway 생성 후 `available` 상태까지 약 60~90초 대기 필수

\$ aws ec2 describe-nat-gateways → State: available 확인 후 다음 진행

1번 발표 | STEP 4-5 EKS 클러스터 + STEP 6 샘플 앱 배포

STEP 4-5 EKS 클러스터 생성

- eksctl create cluster 명령 하나로 컨트롤 플레인 + 노드그룹 한 번에 생성 (약 15~20분 소요)
- t3.medium × 2 · Kubernetes 1.30 · 관리형 노드그룹
- --node-private-networking → 워커노드를 프라이빗 서브넷에 배치 (보안상 필수)

※ --node-private-networking 빠뜨리면 노드가 퍼블릭 서브넷에 올라감 → 보안 위험

\$ kubectl get nodes → 2개 Ready 확인

\$ aws eks update-kubeconfig --name chaos-demo → kubeconfig 등록

\$ kubectl cluster-info → 컨트롤 플레인 엔드포인트 출력되면 연결 완료

STEP 6 샘플 앱 배포

- payment · order 네임스페이스 분리 · nginx:latest · replicas=3
- CPU requests 100m / limits 200m · Memory requests 128Mi / limits 256Mi

\$ kubectl create namespace payment && kubectl create namespace order

\$ kubectl apply -f payment-deployment.yaml && kubectl apply -f order-deployment.yaml

\$ kubectl get pods -n payment → 3개 모두 Running · RESTARTS=0 확인

1번 발표 | STEP 7 Prometheus + Grafana 모니터링

STEP 7 kube-prometheus-stack 설치

- Helm 차트 하나로 Prometheus · Grafana · AlertManager · Node Exporter 전부 설치

- Grafana 관리자 비밀번호: admin1234

```
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
$ helm repo update
```

```
$ helm install monitoring prometheus-community/kube-prometheus-stack #
```

```
$ --namespace monitoring --create-namespace #
```

```
$ --set grafana.adminPassword=admin1234
```

```
$ kubectl get pods -n monitoring → 모든 Pod Running 확인
```

Grafana 접속 및 대시보드 확인

```
$ kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
```

- 브라우저 `http://localhost:3000` → admin / admin1234 로그인
- Dashboards → Kubernetes / Compute Resources / Cluster 선택
- 네임스페이스별 CPU · Memory 사용량 실시간 확인 가능

※ port-forward 실행 중인 터미널은 계속 열어둬야 함 → 닫으면 접속 끊김

1번 발표 | STEP 8 Bedrock + STEP 9 Slack 앱 생성

STEP 8 Amazon Bedrock 모델 설정

- 리전: us-east-1 (버지니아) → Claude Sonnet 4.5 사용
 - API 호출 시 Foundation model ID가 아닌 Inference Profile ID 를 반드시 사용
 - ※ Foundation model ID 직접 사용 시 'on-demand throughput isn't supported' 에러 발생
- ```
$ aws bedrock list-inference-profiles --region us-east-1
```
- \$ 사용할 ID: *us.anthropic.claude-sonnet-4-5-20250929-v1:0*

## STEP 9 Slack 앱 생성

- api.slack.com → Create New App → From scratch → 앱 이름: eks-ops-bot
- OAuth & Permissions → Bot Token Scopes 3개 추가:
  - chat:write · commands · app\_mentions:read
- Socket Mode → Enable → App-Level Token 생성 (Scope: connections:write)
- Slash Commands → /eks 커맨드 등록 (Socket Mode ON이면 Request URL 불필요)
- Install to Workspace → Bot User OAuth Token (xoxb-) 발급
  - ※ xapp- 토큰은 생성 직후에만 전체 값 확인 가능 → 즉시 복사해서 안전한 곳에 저장
- 저장해야 할 토큰 3종: xoxb- (Bot Token) · xapp- (App Token) · Signing Secret

# 1번 발표 | STEP 11 MCP 서버 구축

## STEP 11 MCP 서버 구축

- 패키지 설치: @slack/bolt · @aws-sdk/bedrock-runtime · @kubernetes/client-node · dotenv

```
$ npm install @aws-sdk/client-bedrock-runtime @slack/bolt @kubernetes/client-node dotenv
```

- .env 파일 생성 → 토큰 3종 + AWS 설정 입력

- SLACK\_BOT\_TOKEN=xoxb-...

- SLACK\_APP\_TOKEN=xapp-...

- SLACK\_SIGNING\_SECRET=...

- AWS\_REGION=us-east-1

- BEDROCK\_MODEL\_ID=us.anthropic.claude-sonnet-4-5-20250929-v1:0

- server.js 구조: tools 배열 정의 → InvokeModelCommand → tool\_use agentic loop

```
$ node server.js →  EKS MCP 서버 실행 중 출력되면 성공
```

# 1번 발표 | STEP 12 /eks 커맨드 시연

---

## STEP 12 /eks 커맨드 시연

- Slack 채널에 eks-ops-bot 초대 후 /eks 명령 입력

\$ /eks payment 네임스페이스 Pod 상태 알려줘

\$ /eks order 서비스 배포 상태 알려줘

\$ /eks payment-service 재시작해줘

\$ /eks payment 네임스페이스 전체 헬스체크 해줘 → 멀티툴 agentic 응답



## 2번 발표 | STEP 1 Kiro로 FIS 템플릿 생성 + STEP 2 IAM 역할

### STEP 1 Kiro로 FIS 실험 템플릿 생성

- kiro.dev 접속 → Windows 설치 → AWS Builder ID로 로그인
- New Session → 채팅창에 FIS ExperimentTemplate JSON 생성 프롬프트 입력
- actionId: aws:eks:pod-delete → 반드시 이 값 (aws:eks:terminate-pod 는 오류)
- filters 블록으로 namespace 지정 불가 → parameters 안에 namespace · selectorType · selectorValue 직접 지정
- roleArn은 PLACEHOLDER\_ARN으로 생성 → IAM ARN 확보 후 교체

※ Kiro가 수정 요청을 대화로 처리해줌 → 프롬프트로 iterative하게 다듬기

*\$ notepad C:\eks-demo\#fis-pod-termination.json → 파일로 저장*

### STEP 2 IAM 역할 생성

- IAM → 역할 만들기 → AWS 서비스 → 사용 사례: FIS EKS 선택
- AWSFaultInjectionSimulatorEKSAccess 정책 자동 첨부됨 · 역할명: fis-experiment-role
- 역할 생성 완료 후 ARN 복사 → JSON 파일에서 PLACEHOLDER\_ARN을 실제 ARN으로 교체

*\$ (Get-Content fis-pod-termination.json) -replace 'PLACEHOLDER\_ARN','실제ARN' | Set-Content fis-pod-termination.json*

## 2번 발표 | STEP 3 RBAC 설정 + FIS 템플릿 등록

### STEP 3 Kubernetes RBAC 리소스 생성

- payment 네임스페이스에 ServiceAccount · Role · RoleBinding 생성
- Role 권한: pods get/list/watch/delete · deployments/replicasets get/list

```
$ kubectl apply -f fis-sa-rbac-v2.yaml
```

```
$ kubectl get sa,role,rolebinding -n payment → 3개 모두 생성 확인
```

### FIS 실험 템플릿 최종 수정

- selectionMode: COUNT(1) → 실습 환경에서는 비율보다 개수 지정이 안정적
- parameters에 kubernetesServiceAccount: fis-experiment-sa 추가

### FIS 실험 템플릿 등록

```
$ aws fis create-experiment-template #
```

```
$ --cli-input-json file:///C:/#eks-demo#fis-pod-termination.json #
```

```
$ --region ap-northeast-2
```

※ 출력된 Template ID (EXTxxx)를 반드시 메모 → 이후 Slack 명령어와 .env에서 사용

```
$ aws fis list-experiment-templates --region ap-northeast-2 → 등록 확인
```

## 2번 발표 | STEP 4 IRSA 연결 (IAM Roles for Service Accounts)

### IRSA 설정 4단계 (모두 빠짐없이 해야 FIS가 클러스터 접근 가능)

- ① OIDC Provider 생성

```
$ eksctl utils associate-iam-oidc-provider --cluster chaos-demo --approve
```

- ② 신뢰 정책 JSON 작성 → fis.amazonaws.com + OIDC Federated 두 가지 Principal 포함

```
$ aws iam update-assume-role-policy --role-name fis-experiment-role --policy-document file://fis-trust-irsa.json
```

- ③ ServiceAccount에 IAM 역할 ARN 어노테이션 추가

```
$ kubectl annotate serviceaccount fis-experiment-sa #
```

```
$ eks.amazonaws.com/role-arn=arn:aws:iam::계정ID:role/fis-experiment-role -n payment
```

- ④ EKS Access Entry 등록 + AmazonEKSClusterAdminPolicy 연결

```
$ aws eks create-access-entry --cluster-name chaos-demo --principal-arn 역할ARN --type STANDARD
```

```
$ aws eks associate-access-policy --cluster-name chaos-demo --principal-arn 역할ARN #
```

```
$ --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy #
```

```
$ --access-scope type=cluster
```

※ 4단계 중 하나라도 빠지면 FIS가 클러스터에 접근 불가 → 실험 실패

## 2번 발표 | STEP 5 server.js FIS 통합 + STEP 6 카오스 실험

### STEP 5 server.js에 FIS SDK 통합

- FIS SDK 설치 + server.js 백업

```
$ npm install @aws-sdk/client-fis
```

```
$ Copy-Item server.js server.js.bak
```

- 추가할 tools 3개: run\_fis\_experiment · get\_fis\_experiment\_status · analyze\_chaos\_result
- .env 마지막 줄에 FIS\_TEMPLATE\_ID 추가

```
$ FIS_TEMPLATE_ID=EXT여기에입력
```

```
$ node server.js →  EKS MCP 서버 실행 중 (FIS 통합 완료) 출력 확인
```

### STEP 6 Slack에서 카오스 실험 트리거

- 실험 전 kubectl watch 열어두기

```
$ kubectl get pods -n payment -w
```

- Slack에서 카오스 실험 명령 입력

```
$ /eks payment 네임스페이스에서 카오스 실험 돌려줘. FIS 템플릿 ID는 EXTxxxx야.
```

- 터미널에서 Pod 변화 실시간 관찰: Running → Terminating → ContainerCreating → Running
- Bedrock이 실험 완료 후 자동으로 analyze\_chaos\_result 도구 호출 → 가용성 분석 결과 Slack 응답

## 2번 발표 | STEP 7 Grafana + STEP 8 Q Developer YAML 개선

### STEP 7 Grafana 메트릭으로 셀프힐링 시각화

- 실험 전부터 Grafana를 열어두고 메트릭 변화를 실시간으로 확인

```
$ kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring
```

- Dashboards → Kubernetes / Compute Resources / Namespace (Pods) → namespace: payment
- 실험 전 → 실험 중 CPU 스파이크 → 복구 후 정상화 패턴 확인

### STEP 8 Amazon Q Developer로 HPA · PDB YAML 개선

- VS Code에 Amazon Q 익스텐션 설치 → AWS Builder ID로 로그인 (무료)
- Slack Bedrock 분석 결과를 Q Developer 채팅에 붙여넣고 YAML 생성 요청
- HPA: minReplicas 4 · maxReplicas 10 · CPU 60% 기준 스케일링
- PDB: minAvailable 3 → 동시 중단 최대 1개만 허용

```
$ kubectl apply -f hpa-payment.yaml && kubectl apply -f pdb-payment.yaml
```

```
$ kubectl get hpa -n payment → MINPODS 4 · MAXPODS 10 확인
```

```
$ kubectl get pdb -n payment → MIN AVAILABLE 3 · ALLOWED DISRUPTIONS 1 확인
```

# 리소스 정리 + 꼭 기억할 핵심 포인트

## ⚠ 리소스 정리 (실습 후 즉시 삭제 — NAT GW · EKS 시간당 과금)

- 삭제 순서 준수: EKS 클러스터 → NAT GW (60초 대기) → EIP → IGW → 서브넷 → VPC

```
$ kubectl delete hpa,pdb --all -n payment
```

```
$ eksctl delete cluster --name chaos-demo --region ap-northeast-2 (약 10~15분)
```

```
$ aws ec2 delete-nat-gateway --nat-gateway-id $NAT_2A (2c도 동일)
```

```
$ Start-Sleep -Seconds 60 → aws ec2 release-address --allocation-id $EIP_2A
```

※ 순서 틀리면 DependencyViolation 에러 → 반드시 EKS 먼저 삭제 후 NAT GW

## 핵심 주의사항 요약

- 🔗 Bedrock: Inference Profile ID 사용 → us.anthropic.claude-sonnet-4-5-20250929-v1:0
- ☑ Slack 토큰 3종: xoxb- (Bot) · xapp- (App) · Signing Secret → xapp-는 즉시 복사
- 💡 FIS actionId: aws:eks:pod-delete (aws:eks:terminate-pod 는 오류)
- 🏠 IRSA 4단계: OIDC · 신뢰정책 · SA 어노테이션 · Access Entry 모두 필요